

TP : Optimisation d'un Réseau de Logistique

Première exercice à rendre avant le 19 avril 2026 à 23h59.

Ce travail peut être réalisé seul ou à deux. Un notebook au format `NOM_PRENOM.ipynb` ou `NOM1_PRENOM1-NOM2_PRENOM2.ipynb` est attendu sur Moodle.

N'hésitez pas à commenter votre code et expliquer vos démarches dès que possible. Veillez à rendre un notebook propre, lisible et dont les cellules s'exécutent dans l'ordre.

Note sur l'évaluation : Une part importante de la note sera dédiée à la qualité de vos analyses de performances (benchmarks) ainsi que le choix des métriques.

Contexte du Projet

Vous venez de fonder une entreprise de livraison de colis. Pour tester vos processus à petite échelle, vous installez votre premier entrepôt dans la commune de Saint-Gély-du-Fesc. Votre objectif est de développer le cœur du logiciel de routage de l'entreprise : l'outil qui calculera l'itinéraire de vos chauffeurs.

Au fur et à mesure de votre succès, l'entreprise va s'étendre à toute la métropole, moderniser sa flotte avec des vélos électriques, et devoir gérer des tournées massives. À chaque étape, vous devrez adapter vos algorithmes, évaluer leurs performances statistiques et les valider sur de **vraies données cartographiques**.

Exercice I : Les débuts locaux : De l'entrepôt au client

Votre zone de livraison est restreinte au réseau routier de Saint-Gély-du-Fesc. Lorsqu'un chauffeur part de l'entrepôt (point A) pour livrer un colis à une adresse (point B), votre application doit lui fournir l'itinéraire le plus rapide. Le réseau routier est modélisé sous forme de graphe où les arêtes ont des poids positifs (le temps de trajet).

1. Modéliser le problème.
2. Implémenter un algorithme classique donnant une solution exacte à ce problème de plus court chemin.
3. **Analyse empirique :** Évaluer la complexité temporelle pratique de votre algorithme.
4. **Données réelles :** Utiliser une librairie Python (comme `osmnx`) pour télécharger le vrai réseau routier de Saint-Gély-du-Fesc. Choisir deux intersections réelles (votre entrepôt et un client), exécuter votre algorithme, et afficher le chemin trouvé ainsi que le temps de trajet estimé.

Exercice II : L'expansion métropolitaine : Passage à l'échelle

Face à votre succès, l'entreprise s'étend. Vous livrez désormais depuis le centre de Montpellier jusqu'aux communes environnantes. Le réseau routier passe de quelques centaines à des dizaines de milliers d'intersections. Le calcul de l'itinéraire pour une course longue distance prend trop de temps et sature vos serveurs.

1. Exécuter votre algorithme précédent sur un graphe très dense. Que constatez-vous concernant le nombre de routes explorées dans de mauvaises directions avant de trouver l'adresse de destination ?
2. Vous disposez des coordonnées spatiales (latitude, longitude) de chaque intersection. Proposer et implémenter un nouvel algorithme permettant d'accélérer le calcul en "guidant" la recherche vers le point d'arrivée.
3. **Données réelles et Benchmark spatial/temporel :** Télécharger le réseau routier de toute la métropole de Montpellier. Tirer aléatoirement au moins 50 paires de nœuds (départ/arrivée). Tracer des graphiques comparant statistiquement les deux algorithmes.

Exercice III : Transition écologique : Les vélos-cargos et l'énergie

Vous remplacez vos camionnettes par des vélos-cargos électriques. L'objectif logiciel change : pour éviter la panne, l'itinéraire de livraison ne doit plus minimiser le temps, mais **minimiser l'énergie consommée** par la batterie.

Ces vélos possèdent un freinage régénératif : monter une côte coûte de l'énergie, rouler sur le plat en coûte un peu, mais **descendre une pente recharge la batterie**.

1. Modéliser ce nouveau problème énergétique.
2. Prouver (théoriquement ou via un contre-exemple sur un mini-graphe) pourquoi les algorithmes utilisés aux Exercices I et II peuvent échouer ou donner de mauvais résultats avec ces nouvelles règles physiques.
3. Implémenter un algorithme capable de trouver la solution exacte sur un graphe comportant des poids négatifs.
4. **Surcoût algorithmique** : Comparer empiriquement la complexité de ce nouvel algorithme avec celui de l'Exercice II.
5. **Données réelles** : Télécharger un quartier particulièrement vallonné (ex : le centre historique de Montpellier). Associer un dénivelé (altitude) aux nœuds pour calculer le poids énergétique des rues. Utiliser votre algorithme pour trouver l'itinéraire de livraison le plus économe en énergie entre deux points.

Exercice IV : Le boom des commandes : Optimisation des tournées quotidiennes

Le volume de colis explose. Vos chauffeurs ne font plus des allers-retours pour un seul colis. Un livreur part désormais le matin avec un vélo chargé de N colis à livrer à N adresses différentes, puis doit rentrer à l'entrepôt. L'enjeu est de calculer l'ordre de passage optimal.

1. Modéliser ce problème pour générer une tournée "Express" (minimisant le temps total de trajet de la journée).
2. Implémenter un algorithme *brute-force* pour trouver l'ordre de livraison optimal. Tracer sa courbe de temps d'exécution en fonction de N . À partir de quel N cette approche devient-elle inutilisable en pratique ?
3. Il existe des méthodes plus efficaces que le *brute-force* pour trouver la solution optimale, en tester une (*on ne vous demande pas de la programmer, utilisez des librairies externes*).
4. Même si elles sont plus rapides que le *brute-force*, ces méthodes restent lentes en pratiques pour des instances trop grandes. Proposer et implémenter une méthode heuristique simple (algorithme glouton) permettant de trouver une tournée très rapidement.
5. Proposer et implémenter une méthode d'amélioration locale pour optimiser la tournée générée à la question précédente.
6. **Benchmark Heuristique vs Exact** : Sur des instances générées de différentes tailles, tracer un graphique comparant la *qualité de la solution* (distance totale) de vos heuristiques par rapport à la solution exacte (quand elle est calculable). Le compromis temps/qualité est-il satisfaisant ?
7. **Données réelles** : Choisir un point de départ (entrepôt) et générer 20 adresses de clients aléatoires dans le réseau de Montpellier.
 - Utiliser l'algorithme de l'Exercice II pour calculer la matrice des temps de trajet entre ces 21 points.
 - Utiliser vos heuristiques pour calculer l'ordre optimal de livraison et afficher la tournée finale.